

Módulo 4. Bucles y depuración

Java desde cero · Repetición, trazas y errores frecuentes



Aprenderemos a repetir instrucciones, controlar iteraciones y detectar errores básicos sin avanzar demasiado rápido.

Idea central

Un bucle evita repetir código manualmente y hace que el programa trabaje de forma más automática.

Herramientas

while · do-while · for · contadores · acumuladores · trazas · depurador

Meta docente

Que el alumnado explique qué pasa en cada vuelta del bucle, no solo que copie código.

Objetivos del módulo

Al terminar este bloque, el alumnado debe poder leer, escribir y corregir bucles básicos.

- Repetir procesos con while, do-while y for.
- Controlar iteraciones usando condiciones claras.
- Aplicar contadores y acumuladores.
- Localizar errores mediante trazas y depuración.
- Explicar qué ocurre en cada vuelta del bucle.

```
int contador = 1;

while (contador <= 5) {
    System.out.println(contador);
    contador++;
}
```

Qué es un bucle

Un bucle permite repetir instrucciones sin copiar el mismo código muchas veces.

- Sin bucle: copiaríamos varias instrucciones casi iguales.
- Con bucle: escribimos una estructura que repite por nosotros.
- Cada repetición se llama iteración.
- El bucle termina cuando la condición deja de cumplirse.

```
System.out.println(1);
System.out.println(2);
System.out.println(3);

// Mejor:
for (int i = 1; i <= 3; i++) {
    System.out.println(i);
}
```

Cuándo usar un bucle

La clave no es memorizar sintaxis, sino detectar cuándo hay una repetición.

Repetir un número fijo

Mostrar una tabla del 1 al 10. Normalmente usamos for.

Repetir hasta que pase algo

Pedir una contraseña hasta que sea correcta. Suele encajar while o do-while.

Recorrer datos

Pasar por notas, productos o nombres de una lista. Más adelante se usará mucho.

```
// Pregunta útil:  
// ¿Sé cuántas veces se repite?  
  
// Sí -> for  
// No -> while / do-while
```

while: repetir mientras se cumpla

while evalúa la condición antes de entrar. Si la condición es falsa, no se ejecuta ninguna vez.

- Primero se comprueba la condición.
- Si la condición es verdadera, entra al bloque.
- Después vuelve a comprobar la condición.
- Si la condición es falsa desde el principio, no entra nunca.

```
int contador = 1;

while (contador <= 5) {
    System.out.println(contador);
    contador++;
}

// Muestra: 1 2 3 4 5
```

Partes de un while

Todo while necesita una variable de control que cambie dentro del bucle.

1. Inicio

Variable que empieza el proceso.
Ejemplo: contador = 1

2. Condición

Pregunta que decide si seguimos.
Ejemplo: contador <= 5

3. Cambio

Modificación que acerca al final. Ejemplo:
contador++

```
int contador = 1;           // inicio

while (contador <= 5) {    // condición
    System.out.println(contador);
    contador++;            // cambio
}
```

do-while: ejecutar al menos una vez

do-while primero ejecuta y después comprueba la condición.

- La instrucción se ejecuta una vez obligatoriamente.
- Después se evalúa la condición.
- Se usa mucho en menús: mostrar opciones y pedir respuesta.
- Cuidado: aunque la condición sea falsa, entra una vez.

```
int opcion;  
  
do {  
    System.out.println("1. Continuar");  
    System.out.println("0. Salir");  
    opcion = 0;  
} while (opcion != 0);
```

for: repetición controlada

for es cómodo cuando sabemos cuántas veces queremos repetir.

- El for reúne inicio, condición y cambio en una sola línea.
- Es ideal cuando conocemos el número de vueltas.
- La variable i suele usarse como contador.
- La lectura correcta es: desde 1, mientras i sea menor o igual que 5, suma 1.

```
for (int i = 1; i <= 5; i++) {  
    System.out.println(i);  
}
```

```
// i = 1    entra  
// i = 2    entra  
// ...  
// i = 6    termina
```

Comparación rápida

while, do-while y for hacen repeticiones, pero se usan en situaciones distintas.

while

Cuando no siempre sabemos cuántas vueltas habrá. Ejemplo: repetir hasta acertar.

do-while

Cuando el bloque debe ejecutarse al menos una vez. Ejemplo: menú.

for

Cuando sabemos el inicio, el final y el avance. Ejemplo: tabla de multiplicar.

```
while (condicion) { ... }  
  
do { ... } while (condicion);  
  
for (inicio; condicion; cambio) { ... }
```

Contadores

Un contador guarda cuántas veces ocurre algo o en qué vuelta estamos.

- Empieza normalmente en 0 o 1.
- Se incrementa con contador++ o contador = contador + 1.
- Permite controlar vueltas, intentos o elementos procesados.
- Debe cambiar dentro del bucle si controla la condición.

```
int intentos = 0;

while (intentos < 3) {
    System.out.println("Intento " +
        intentos);
    intentos++;
}

System.out.println("Fin");
```

Acumuladores

Un acumulador guarda un total que se va construyendo poco a poco.

- Empieza con un valor neutro, por ejemplo 0.
- En cada vuelta se suma, resta o acumula un cálculo.
- Se usa para totales, medias, importes o puntuaciones.
- Al final se muestra el resultado acumulado.

```
int suma = 0;

for (int i = 1; i <= 100; i++) {
    suma += i;
}

System.out.println("Suma: " + suma);
```

break y continue

Son instrucciones útiles, pero deben usarse con cuidado para no ocultar la lógica.

- break termina el bucle inmediatamente.
- continue salta a la siguiente vuelta.
- No conviene abusar: a veces hacen el código menos claro.
- Antes de usarlos, pregunta si la condición del bucle puede expresarlo mejor.

```
for (int i = 1; i <= 10; i++) {  
    if (i == 5) {  
        continue;  
    }  
  
    if (i == 8) {  
        break;  
    }  
  
    System.out.println(i);  
}
```

Bucle infinito

Un bucle infinito ocurre cuando la condición nunca llega a ser falsa.

- La condición permanece siempre verdadera.
- La variable de control no cambia o cambia mal.
- El programa parece quedarse bloqueado.
- Para detenerlo en VS Code: botón de parar o Ctrl + C en terminal.

```
int contador = 1;

while (contador <= 5) {
    System.out.println(contador);
    // Falta contador++;
}

// Este bucle no termina.
```

Trazas de depuración

Una traza es un mensaje temporal para observar qué vale una variable.

- Sirven para ver qué está pasando dentro del programa.
- No son la solución final: se usan para investigar.
- Permiten comprobar el valor de una variable en cada vuelta.
- Después de corregir, se pueden quitar o comentar.

```
int suma = 0;

for (int i = 1; i <= 5; i++) {
    suma += i;
    System.out.println("i=" + i + " suma=" +
        suma);
}

System.out.println("Resultado: " + suma);
```

Depurar en VS Code

El depurador permite ejecutar paso a paso y ver el valor de las variables.

Breakpoint

Punto rojo donde el programa se detiene.

Step Over

Ejecuta la siguiente línea sin entrar en métodos.

Variables

Panel donde vemos cómo cambian los valores.

- Pasos recomendados:
- 1. Poner un punto de interrupción.
- 2. Ejecutar en modo Debug.
- 3. Avanzar línea a línea.
- 4. Observar cómo cambian contador, suma o intento.

Ejemplo guiado 1

Mostrar números del 1 al 5 con while.

- Creamos un contador que empieza en 1.
- Mientras sea menor o igual que 5, se muestra.
- Después se incrementa.
- Cuando llega a 6, la condición es falsa y termina.

```
public class ContadorWhile {  
    public static void main(String[] args) {  
        int contador = 1;  
  
        while (contador <= 5) {  
            System.out.println(contador);  
            contador++;  
        }  
    }  
}
```

Ejemplo guiado 2

Mostrar una tabla de multiplicar con for.

- La variable numero contiene la tabla que queremos mostrar.
- El for va de 1 a 10.
- En cada vuelta calcula numero * i.
- printf permite dar formato a la salida.

```
public class TablaMultiplicar {  
    public static void main(String[] args) {  
        int numero = 7;  
  
        for (int i = 1; i <= 10; i++) {  
            System.out.printf("%d x %d = %d%n",  
                              numero, i, numero * i);  
        }  
    }  
}
```

Ejemplo guiado 3

Sumar números del 1 al 100 usando un acumulador.

- Inicializamos suma en 0.
- Recorremos los números del 1 al 100.
- En cada vuelta añadimos i a suma.
- Al finalizar, suma contiene el total.

```
public class SumaNumeros {  
    public static void main(String[] args) {  
        int suma = 0;  
  
        for (int i = 1; i <= 100; i++) {  
            suma += i;  
        }  
  
        System.out.println("Suma total: " + suma);  
    }  
}
```

Ejercicio 1

Tabla de multiplicar: pedir un número y mostrar del 1 al 10.

- Crear una clase llamada TablaMultiplicar.
- Pedir o definir un número entero.
- Mostrar la tabla del 1 al 10.
- Usar un bucle for.
- Comprobar que aparecen 10 líneas.

```
/*  
Enunciado:  
Crea un programa que guarde un número entero  
y muestre su tabla de multiplicar del 1 al  
10.  
*/  
  
public class TablaMultiplicar {  
    public static void main(String[] args) {  
        int numero = 5;  
        // Completar con un for  
    }  
}
```

Ejercicio 2

Adivina el número: repetir intentos hasta acertar.

- Generar un número secreto entre 1 y 100.
- Pedir intentos hasta acertar.
- Indicar si el intento es mayor o menor.
- Usar do-while porque al menos se necesita un intento.
- Más adelante se puede añadir contador de intentos.

```
int secreto = (int)(Math.random() * 100) + 1;
int intento;

do {
    // leer intento
    if (intento < secreto) {
        System.out.println("Más alto");
    } else if (intento > secreto) {
        System.out.println("Más bajo");
    }
} while (intento != secreto);
```

Errores frecuentes

Los errores en bucles suelen estar relacionados con condiciones, llaves o variables que no cambian.

Falta ;

Aparece mucho al escribir instrucciones dentro del bucle.

Llaves mal cerradas

El bloque del bucle termina donde no debería.

Variable no cambia

Provoca bucles infinitos.

```
while (contador <= 5) {  
    System.out.println(contador) // falta ;  
    // falta contador++;  
}
```

```
// También revisar:  
// mayúsculas, nombres y llaves { }
```

Buenas prácticas

Antes de ejecutar, conviene leer el bucle como una frase en castellano.

- Usar nombres claros: contador, suma, intento, numero.
- No mezclar muchas tareas dentro del mismo bucle.
- Comprobar primero con pocos valores: 1 a 5 antes que 1 a 100.
- Explicar el bucle en voz alta antes de ejecutarlo.
- Formatear el código para ver bien las llaves.

```
// Buena lectura:  
// Desde i = 1, mientras i <= 10, aumenta i.  
  
for (int i = 1; i <= 10; i++) {  
    System.out.println(i);  
}
```

Mini reto de repaso

Analizar un bucle y predecir qué mostrará antes de ejecutarlo.

```
int x = 2;

while (x < 10) {
    System.out.println(x);
    x = x + 2;
}

System.out.println("Fin");
```

- Antes de ejecutar, responde:
- ¿Qué valor tiene x al empezar?
- ¿Cuántas vueltas dará?
- ¿Qué números se muestran?
- ¿Qué valor hace que el bucle termine?

Checklist final

Lista de comprobación antes de dar por cerrado el módulo.

- Puedo explicar qué es una iteración.
- Sé cuándo usar while, do-while o for.
- Sé crear un contador.
- Sé crear un acumulador.
- Sé identificar un bucle infinito.
- Sé usar trazas para revisar valores.
- Puedo leer un bucle y predecir su salida.

```
// Frase clave del módulo:  
// Un bucle repite  
instrucciones  
// mientras una condición se  
cumple.
```

```
// La condición debe cambiar  
// para que el bucle pueda  
terminar.
```

Cierre del módulo

Después de bucles, el siguiente paso natural son métodos y descomposición.

- El alumnado ya ha practicado sintaxis, variables, operadores, condicionales y bucles.
- El siguiente módulo debe enseñar a dividir el programa en partes más pequeñas.
- Los métodos ayudarán a evitar código largo y repetido.
- Conviene cerrar este bloque con 2 ejercicios corregidos en directo.

```
public static void  
mostrarTitulo() {  
    System.out.println("Mi  
programa");  
}  
  
// Próximo módulo:  
// métodos, parámetros y  
retorno.
```

Basado en el Módulo 4 del manual: Bucles y depuración.